POrTAL: Plan-Orchestrated Tree Assembly for Lookahead

Evan Conway
University of Virginia
Charlottesville, Virginia
auj4kq@virginia.edu

David Porfirio
George Mason University
Fairfax, Virginia
dporfiri@gmu.edu

David Chan
Navy Center for Applied Research in AI
US Naval Research Laboratory
Washington, D.C.
david.j.chan4.civ@us.navy.mil

Mark Roberts
Navy Center for Applied Research in AI
US Naval Research Laboratory
Washington, D.C.
mark.c.roberts20.civ@us.navy.mil

Laura M. Hiatt
Navy Center for Applied Research in AI
US Naval Research Laboratory
Washington, D.C.
laura.m.hiatt.civ@us.navy.mil

***Abstract*— Assigning tasks to robots often involves supplying the robot with an overarching *goal*, such as through natural language, and then relying on the robot to uncover and execute a *plan* to achieve that goal. In many settings common to human-robot interaction, however, the world is only partially observable to the robot, requiring that it create plans under uncertainty. Although many probabilistic planning algorithms exist for this purpose, these algorithms can be inefficient if executed with the robot's limited computational resources, or may require more steps than expected to achieve the goal. We thereby created a new, lightweight, probabilistic planning algorithm, *Plan-Orchestrated Tree Assembly for Lookahead (POrTAL)*, that combines the strengths of two baseline planning algorithms, FF-Replan and POMCP. In a series of case studies, we demonstrate *POrTAL's* ability to quickly arrive at solutions that outperform these baselines in terms of number of steps. We additionally demonstrate how *POrTAL* performs under varying temporal constraints.**

I. Introduction

The ability of modern robots to respond to arbitrary user requests has advanced considerably in recent years. This advancement is in large part due to robots' ability to autonomously plan their own actions. When receiving a goal such as "bring me a cup of coffee," for example, a robot can calculate the minimum number of steps required to achieve this goal: obtain the coffee grinds, proceeding to the coffee maker, load the grinds, and so on.

In many scenarios common to human-robot interaction, however, this planning must be performed under considerable uncertainty. Such uncertainty can arise from uncertainty over the outcome of the robot's actions or uncertainty in the environment. This paper focuses on the latter case. Specifically, task-critical entities in the robot's environment may have uncertain locations, such as if the location of the coffee grinds is unknown. In this case, the robot may still possess a distribution of possible entity locations, sourced from prior exploration of its surroundings or provided directly from the end user [12]. The robot must then create a plan that adheres to some objective function, which may be to minimize the *expected* number of steps to achieve the goal, or maximize the *expected* reward that it receives along the way. Fortunately, many task *anytime* planning approaches exist for this purpose, such as FF-Replan [16] and Partially Observable Monte Carlo Planning (POMCP) [14], which enable the robot to continuously improve its plan while it executes its task.

Choosing a planning algorithm can drastically affect the outcome of a task. Consider the household delivery robot shown in Figure 2 tasked with delivering a cup and a plate to the kitchen table. Suppose the robot knows that the cup has an 80% chance of being in the bathroom and a 20% chance of already being in the kitchen. The robot must decide where to look first—should it guess that the cup will be in the bathroom, where the cup is most likely to be? If the cup is not in the bathroom, the cost of backtracking to its starting point, from which it can proceed to the kitchen, is high. Alternatively, given that the robot is already so close to the kitchen, should the robot quickly check the kitchen? Although the cup is less likely to be in the kitchen, the cost of backtracking is much lower.

In terms of minimizing expected number of plan steps, FF-Replan performs well in practice [9], but is not optimal. While Monte Carlo approaches such as POMCP maximizes the robot's expected reward, they struggle without a strong reward signal, such as if the robot only receives reward from achieving its goal. Furthermore, these approaches require the rapid construction of large search trees, which is computationally intensive, and potentially even prohibitive if computation is performed solely onboard the robot.

We thereby created a new anytime planning algorithm, *Partially Observable Lookahead Planning (POrTAL)*. *POrTAL* works by combining the strengths of both POMCP and FF-Replan. Like POMCP, *POrTAL* iteratively generates a search tree during task execution. Unlike POMCP, *POrTAL* utilizes a classical planner to limit the breadth-

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

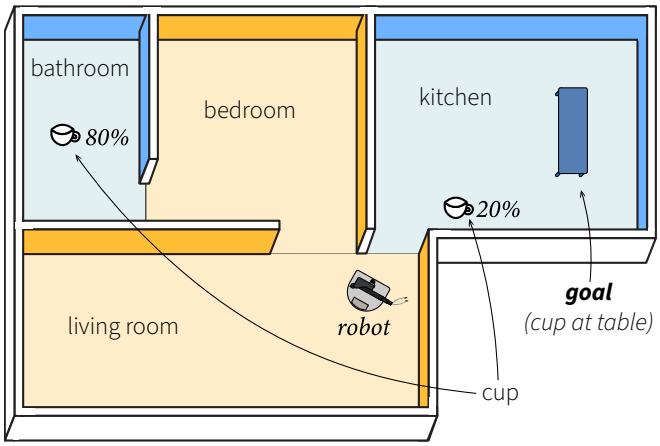

Fig. 1. Simple household environment in which a robot is tasked to deliver a cup to the kitchen table. The cup has a 20% chance of being in the kitchen already, and an 80% chance of being in the bathroom. Although there is a smaller chance of the cup being in the kitchen, a policy that minimizes the number of anticipated steps to deliver the cup may dictate that the robot check in the kitchen first.

wise expansion of the tree and more rapidly facilitate its depth-wise expansion towards the goal. To expand the tree depth-wise, *POrTAL* draws inspiration from FF-Replan by sampling a determinized variant of the problem, using a classical planner to generate a linear plan, and inserting the entire plan into the tree at once as a new branch. Inclusive of the root, treenodes that might diverge from the expectations of the classical plan are labeled as *interesting* and marked for further exploration. We compared *POrTAL* to POMCP and FF-Replan and found that *POrTAL* often produces plans of lower length for scenarios that are common to human-robot interaction.

Our contributions are as follows:

- *Technical*—The *POrTAL* algorithm, which balances efficiency and plan length for realistic human-robot interaction tasking scenarios.
- *Empirical*—An evaluation of *POrTAL* against FF-Replan and POMCP baselines.

II. Background

We provide a brief background in automated planning, followed by several existing approaches for online probabilistic planning.

A. Automated Planning

In decision making for artificial intelligence (AI), automated planning is the process of deciding what actions an agent needs to perform [4]. Within the scope of task planning, classical planning involves the robot being given a deterministic domain and a goal to achieve, which the robot uses to plan its actions prior to task execution. A key limitation of classical planners is the assumption that all object locations are known to the robot. This assumption is often not the case, such as in households [11] or search-and-rescue scenarios [7] in which the locations of task-critical entities are unknown or change frequently.

Instead, the robot will have only prior knowledge of the likelihood of object locations, which may be provided from the robot's past experiences, or by-hand from a human counterpart [12].

B. Probabilistic Planning

We build on two approaches for online probabilistic planning that alternate between planning and execution stages [8].

a) FF-Replan: FF-Replan describes an approach for online probabilistic planning in which the domain is determinized prior to execution [16]. FF-Replan then computes the optimal (*i.e.,* shortest path) plan for that determinization and follows the plan until it fails, after which it replans from that point. The result is a branching plan. Due to its historical success as a good heuristic [9], FF-Replan influences our design of *POrTAL*, specifically in the use of determinization and subsequent invocation of classical planning. More information about FF-Replan is provided in Section §III-A.1.

Still, FF-Replan suffers from a few notable drawbacks. First, it does not improve with greater compute time, whereas other approaches (*e.g.,* POMCP) produce better results if the algorithm is allowed to run for greater durations. A second disadvantage of FF-Replan is that it does not incorporate contingencies in computing a plan for a particular determinization, or points in the plan in which the robot's beliefs are found to mismatch the true state of the world. In our simple delivery example (Figure 1), consider the case in which the robot determinizes the world such that the cup is believed to be in the bathroom. The robot might subsequently (1) create a plan to fetch the cup from the bathroom, and then at runtime, (2) find that the cup is not in the bathroom and (3) backtrack to the living room so that it can search for the cup in the kitchen. A better solution would have been for the robot to create a plan that balances the probability of the cup existing in a particular location with the anticipated cost of backtracking from this location. Thus, the best plan (which FF-Replan does not produce) would be for the robot to first check the kitchen, knowing that the cost of backtracking from the kitchen is low.

b) POMCP: Partially Observable Monte Carlo Planning (POMCP) is an anytime probabilistic planning algorithm for Partially Observable Markov Decision Processes (POMDPs) that iteratively constructs a search tree over all possible paths in a plan and explores different paths based on expected reward [14]. POMCP is a *partially observable* variant of Monte Carlo Tree Search (MCTS) [1]. Similar to MCTS, each iteration is comprised of a *simulation* through the existing tree space, which ends in a *rollout* through unexplored tree space. In contrast to MCTS, however, POMCP operates over belief states resulting from uncertainty in the environment and from the robot's actions.

Rather than planning to achieve a goal like FF-Replan, both MCTS and POMCP strive to achieve an optimal *policy* over a reward space. Rewards received during rollouts are propagated up the tree, thus weighing certain paths in the

tree more heavily than others. We incorporate aspects of POMCP into our design of *POrTAL*—specifically the iterative construction of a search tree—because of its performance guarantees. More information about POMCP is provided in Section §III-A.2.

Although POMCP converges to an optimal policy, it struggles along two dimensions. First, upon encountering a portion of the search tree that has not yet been explored (such as the root at the beginning of search) POMCP initially examines *all* possible paths to a goal. Only after multiple simulations will more favorable paths be weighted more highly than others. As a result, POMCP can require substantial effort to narrow its search. This issue is compounded as the number of possible actions for the robot to perform at any given step increases, which increases the breadth of its search. Second, convergence can be slowed under sparse rewards, such as if reward is only received when the goal is achieved. In sufficiently complex problems with long planning horizons, few rollouts will receive any reward, drastically increasing the sample complexity.

c) Other approaches: Beyond online tree search approaches like MCTS and POMCP, more recent probabilistic planning methods aim to address scalability and model generalization in partially observable environments. One such approach is IntentMCTS, which extends MCTS by incorporating inferred multi-step human intent via a memory module and reward augmentation, adapting tree expansion and action selection to account for human-controlled decision points [2]. DESPOT (Determinized Sparse Partially Observable Tree) is an anytime POMDP planning algorithm that balances search accuracy and efficiency by sampling a small set of representative scenarios from the belief distribution and constructing a sparse belief tree [15]. By using upper and lower bounds to prune the search space, DESPOT achieves strong performance guarantees while remaining computationally feasible for complex problems. In contrast, QMDP-Net represents a class of methods that embed planning algorithms directly into differentiable neural architectures [6], thereby leveraging the strengths of model-based planning and deep learning. It approximates QMDP [10], an algorithm that simplifies POMDPs by assuming full observability after one step, within a neural network. This enables end-to-end training from data while retaining an interpretable planning structure.

III. Technical Approach

POrTAL draws on both FF-Replan and POMCP. In what follows, we provide preliminary information on FF-Replan and POMCP. Then, we describe *POrTAL* in detail.

A. Preliminaries

1) FF-Replan: Traditionally, FF-Replan has been applied to MDPs, where uncertainty arises from nondeterministic action effects, yet with a fully observable world. Instead, we are interested in the inverse problem—robots with deterministic action effects and reliable sensors, but with an uncertain initial belief about the world. POMDPs are therefore an appropriate representation. A partially observable Markov decision process (POMDP) is a tuple $(S, A, T, R, \Omega, O, \gamma)$ where S is a set of states, A a set of actions, $T(s' \mid s, a)$ the state transition probability, $R(s, a)$ the reward function, Ω a set of observations, $O(o \mid s', a)$ the observation probability, and $\gamma \in [0, 1)$ the discount factor.

Our task is to apply FF-Replan to these POMDPs. To do so, we first determinize the problem by choosing a *particular state* s from the current belief $\mathcal{B}(h)$. A particle refers to a sample determinization, while the belief refers to the set of particles that represent all possible true states at history h, which is the sequence of robot actions and observations $a_0 o'_0 ... a_n o'_n$ over time. We apply the strategy used in [12], which is to choose the state s that occurs with the highest probability. We then use a planner $\mathcal{P}$ to find a plan $a_0, ..., a_n$ for the state s. Using our starting state s and simulator $\mathcal{G}$, we can then find the expected observations after performing each action from state s. This gives us an expected history $a_0 o'_0 ... a_n o'_n$.

Following the expected history, we can apply each action $a_0, ..., a_n$ in sequence and receive a true observation o_t after each action a_t. When a true observation o_t differs from its corresponding expected observation o'_t, we replan from the history $h a_0 o_o ... a_t o_t$. The role of a history in FF-Replan for POMDPs is analogous to the role of a state in traditional FF-Replan: we plan from a particular history/state, and replan as soon as we enter a history/state that we did not expect.

A limitation of FF-Replan is that choosing the state s that occurs with the highest probability can lead to very poor plans. For example, consider a situation where there is a chance of being in a state where the robot can quickly complete the task, and a slightly larger chance of being in a state where the robot needs a long time to complete the task. If we only select the most probable state, then we would ignore the possibility of completing the task far more rapidly. Other heuristic strategies, such as choosing the state that yields the shortest plan, have similar issues.

2) POMCP: POMCP uses a tree structure to represent possible future histories. The root represents the current history $a_0 o_0 ... a_t o_t$, and nodes in the tree represent various histories $a_0 o_0 ... a_t o_t a_{t+1} o_{t+1} ....$ We will refer to nodes corresponding to histories ending in an observation as *observation nodes* and nodes corresponding to histories ending in an action as *action nodes*.

POMCP also suggests using particles to approximate the belief state instead of explicitly computing the belief state. This is advantageous because it improves scaling for large belief states, and requires only a black-box simulator $\mathcal{G}$. This simulator call $(s', o, r) \sim \mathcal{G}(s, a)$ takes in a state and an action, and outputs a new state, observation, and reward, according to the transition probabilities for the POMDP. This removes the need to explicitly compute the POMDP transition probabilities.

The initial particles representing the initial belief state are randomly sampled from the initial belief given by the POMDP description, $\mathcal{I}$. To then find the belief for a history $a_0 o_0 ... a_n o_n$ (or hao for short), we would pass each particle

Algorithm 1 *POrTAL* Main Loop

procedure SEARCH(h)
  repeat
    $s \sim \mathcal{B}(h)$
    SIMULATE(s, h)
  until TIMEOUT()
  return $\text{argmax}_b V(hb)$
end procedure

$s \in \mathcal{B}(h)$ into our simulator $\mathcal{G}(s, a)$ and keep only the resulting states s' with matching observations o.

B. POrTAL

Like POMCP, POrTAL uses a history tree representation, with particles at each node representing the belief. POrTAL uses an FF-Replan-like process to generate plans from various points in the tree, which are then added to the tree. This allows POrTAL to focus on promising sequences of actions instead of considering all actions, at the cost of potentially ignoring better actions.

Algorithm 1 depicts *POrTAL*'s main loop. Each search iteration begins by sampling a state $s \sim \mathcal{B}(h)$. SIMULATE then simulates following that particle through the tree, using UCT for action selection and $\mathcal{G}$ to obtain the resulting observations (Algorithm 2). At some observation node along this path, we will halt, plan, and call ROLLOUT on that plan (Algorithm 3).

Our choice of when to halt is influenced by two considerations. The first consideration is that we only want to generate plans from a subset of observation nodes, which we will mark as being *meaningful*. These nodes represent points where our plan has failed and we need to replan. The second consideration is that we would like to balance between generating plans from nodes higher up in the tree (potentially finding new actions to consider) and generating plans from nodes further down in the tree (allowing for more accurate evaluation of our action values). To determine whether or not to generate a plan at a specific node, we will rely on EXPAND, which takes in the number of visits and plans generated for a particular node, and outputs whether or not to halt at that node.

When adding a plan to the tree in ROLLOUT, POrTAL simulates performing that plan on *all* particles $s_{alt} \in \mathcal{B}(h)$, stopping when an observation is encountered that differs from the observation particle s had at the same time step. The observation nodes corresponding to these deviations are marked as *meaningful*, and represent nodes where we wish to replan. This process is equivalent to simulating the first cycle of FF-Replan for POMDPs for all of the particles.

While adding a plan in SIMULATE and ROLLOUT, we simultaneously estimate the value of each node. The value of an observation node is the return of the best action available at that node, so $V(h) = \max_b V(hb)$. If an observation node has no children, we instead estimate its return using the approximation $V(h) = \text{APPROX}(h)$. The value of an action node is the sum of the expected immediate reward and the discounted expected future return, $V(ha) = \mathbb{E}[r_t + \gamma \cdot G_t \mid a_t = a] = \mathbb{E}[r_t \mid a_t = a] + \gamma \cdot \mathbb{E}_o[V(hao) \mid a_t = a]$.

When SEARCH runs out of time, the action a_t with the highest value $V(ha_t)$ is chosen as the next action to take. The agent then performs that action and receives a real observation o_t, with $T(ha_to_t)$ becoming the new root node.

IV. EVALUATION

We conducted two evaluations of *POrTAL*. Our first is an *Algorithm Comparison*, in which we compare *POrTAL*'s performance to two baselines, FF-Replan and POMCP. Our second is a *Time Comparison*, in which we compare *POrTAL*'s performance to itself across several different *timeout* values. We define timeout as the amount of time that the robot takes to complete an action, which we use here to define the amount of time that the robot is *allowed* to plan per action. When the timeout is reached, the robot will select the action that has the maximum value at the moment when the time ran out.

A. Domains

Within each of our evaluations, we tested *POrTAL*'s performance in two separate planning domains—an *office* domain and a multi-floor *elevator* domain. Each domain consists of *regions*, *surfaces*, *items*, and possibly *people*. Surfaces and people exist within regions, and items can be grabbed and placed on top of surfaces or on the region floor. Items can also be given to and received from people. The location of items and people in each domain are initially uncertain. The robot can take *search* actions to uncover items at its current location.

The robot can move within different regions and approach each surface. Regions and surfaces are further divided into *cells* in order to indicate where the robot exists inside of a region and, if the robot is close to a surface, what part of the surface is close to. The robot uses a move action to traverse between adjacent cells. The known aspects of

Algorithm 2 *POrTAL* Simulate Procedure

procedure SIMULATE(s, h)
  $N(h) \leftarrow N(h) + 1$
  if $h \in \mathcal{M}$ and EXPAND($N(h), G(h)$) **then**
    $G(h) \leftarrow G(h) + 1$
    $(a_0, \ldots, a_n) \sim \mathcal{P}(s)$
    ROLLOUT($s, h, (a_0, \ldots, a_n)$)
  else
    $a \leftarrow \text{argmax}_b V(hb) + c\sqrt{\frac{\log N(h)}{N(hb)}}$
    $N(ha) \leftarrow N(ha) + 1$
    $(s', o, r) \sim \mathcal{G}(s, a)$
    $V(ha) \leftarrow V(ha) - \frac{|\mathcal{B}(hao)|}{|\mathcal{B}(h)|} \cdot \gamma \cdot V(hao)$
    SIMULATE(s', hao)
    $V(ha) \leftarrow V(ha) + \frac{|\mathcal{B}(hao)|}{|\mathcal{B}(h)|} \cdot \gamma \cdot V(hao)$
    $V(h) \leftarrow \max_b V(hb)$
  end if
end procedure

Algorithm 3 *POrTAL* Rollout Procedure

procedure ROLLOUT$(s, h, (a_k, \ldots, a_n))$
  $(s', o, r) \sim \mathcal{G}(s, a_k)$
  if $ha_k \notin T$ **then**
    $T(ha_k) \leftarrow \langle 0, 0 \rangle$
    for $s_{alt} \in \mathcal{B}(h)$ **do**
      $(s'_{alt}, o_{alt}, r_{alt}) \sim \mathcal{G}(s_{alt}, a_k)$
      if $ha_k o_{alt} \notin T$ **then**
        if $o_{alt} \neq o$ **then**
          $\mathcal{M} \leftarrow \mathcal{M} \cup \{ha_k o_{alt}\}$
        end if
        $T(ha_k o_{alt}) \leftarrow \langle 0, 0, \text{APPROX}(ha_k o_{alt}), \emptyset \rangle$
      end if
      $\mathcal{B}(ha_k o_{alt}) \leftarrow \mathcal{B}(ha_k o_{alt}) \cup \{s'_{alt}\}$
      $V(ha_k) \leftarrow V(ha_k) + \frac{1}{|\mathcal{B}(h)|} \cdot r_{alt}$
    end for
  end if
  $V(ha_k) \leftarrow V(ha_k) - \frac{|\mathcal{B}(ha_k o)|}{|\mathcal{B}(h)|} \cdot \gamma \cdot V(ha_k o)$
  ROLLOUT$(s', ha_k o, (a_{k+1}, \ldots, a_n))$
  $V(ha_k) \leftarrow V(ha_k) + \frac{|\mathcal{B}(ha_k o)|}{|\mathcal{B}(h)|} \cdot \gamma \cdot V(ha_k o)$
  $V(h) \leftarrow \max_b V(hb)$
end procedure

each domain (including regions and surfaces) are defined in the Planning Domain Definition Language (PDDL) [3], including a goal for the robot to achieve. The uncertain aspects of each domain, specifically item locations, are given to the robot as random distributions (see Section §IV-C).

a) Office: Our *office* domain consists of a standard workplace environment with various regions and surfaces. In total, the domain consists of 42 region cells and 29 surface cells. The regions cells consist of both rooms and hallways, while the surface cells consist of tables clustered into each room. The robot's goal is to place both a *cake* and a *cube* in a *box*. The *locations*, or cells that the *cake*, *cube*, and *box* exist in, are all uncertain.

The main source of complexity in this domain is that the task can be completed in multiple different orders: for example, the robot can place the cake in the box and then the cube, or vice versa. Additionally, the robot has one gripper, meaning that it can carry one item at a time or both items within the box, but not two items outside of the box at the same time. For example, the robot can transport both cake and box by placing the cake in the box and then carrying the box, but can not transport both the cake and the cube at the same time. This means that the robot has to consider which items are best to search for earlier on in its task.

b) Elevator: Our *elevator* domain consists of two identical floors joined by an elevator. We model the elevator shaft as a region in vertical space consisting of 10 cells. Thus, moving from one floor to another requires 10 consecutive move actions, representing the time necessary for the robot to successfully move into the elevator, travel to the other floor, and then move out of the elevator. Each floor consists

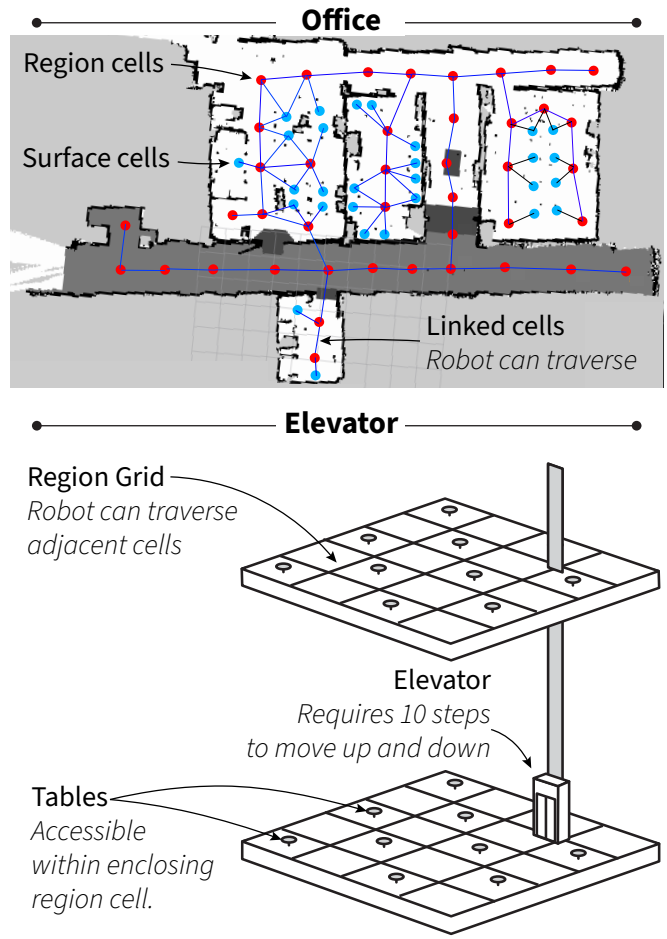

Fig. 2. A visual depiction of both domains used in our evaluation. The *office* domain (top) consists of an office environment with rooms (white) and hallways (grey). Red nodes are region cells and blue nodes are surface cells. Adjacent cells are linked based on the robot's ability to travel between them. The *elevator* domain consists of two floors of a building linked by an elevator. Each floor is divided into a 5x4 grid of region cells that the robot can travel between. Some region cells have surface cells inside of them.

of 20 cells, arranged in a 5 by 4 grid, with the elevator in the lower middle cell. Within each floor, 10 tables are arranged in a checkerboard pattern, with each table being accessible only from a single cell. The robot's goal is to fetch a *delivery*, deliver it to the *recipient*, and then check in with a *staff member*. The locations of the *delivery*, *recipient*, and *staff member* are all uncertain.

The primary source of complexity in this domain arises from the robot needing to choose when to take the elevator to a different floor as opposed to staying on the same floor. Since riding the elevator takes 10 actions each way, traversing to the other floor is costly. The robot needs to be strategic in terms of when it should take the elevator in order to minimize plan length.

B. POrTAL and Baseline Configuration

Each planning algorithm operates on a continuous action-observation loop until the goal is achieved. For *POrTAL* and POMCP, after an action is taken and the robot makes an observation, each algorithm finishes the current loop iteration

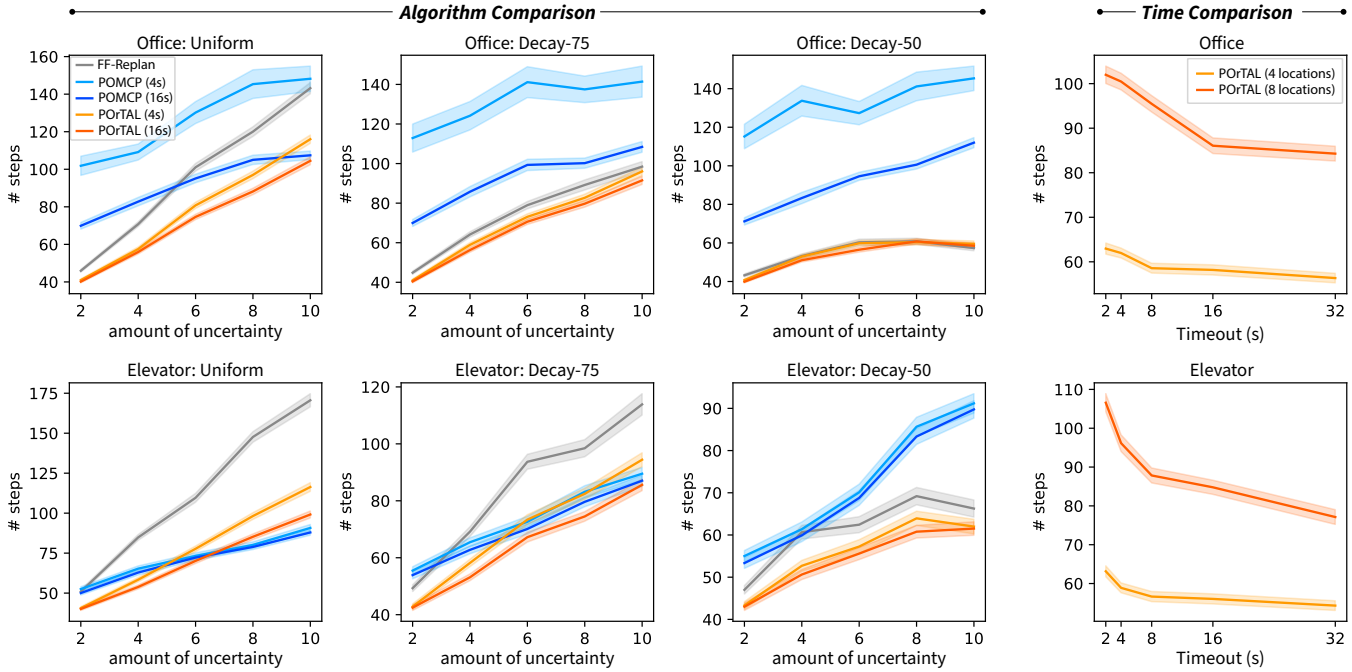

Fig. 3. The results of our *Algorithm Comparison* (left) and *Time Comparison* (right). Error bands represent standard error of the mean.

of the SEARCH procedure before shifting the root of the history tree. Both algorithms are allotted five actions worth of planning time at the very start to represent startup cost, in which an initial tree is constructed before the robot takes any actions. FF-Replan is not given any time restriction, as the amount of time required for it to generate a plan is negligible in these problems.

Preliminary evaluations with POMCP indicated that the algorithm struggles without a strong reward signal. Thus, to ensure a fairer comparison with *POrTAL*, POMCP is given a reward of 1.0 for accomplishing *subgoals* in each domain. In both domains, this reward is given for successfully finding an uncertain item. Subgoal reward is also given for achieving a *landmark* [5], [13], which in the office domain is placing the cake or cube into the box, and in the elevator domain is giving the delivery to the recipient or checking in with the staff member. POMCP is also given a reward of 0.1 for each search action taken by the robot, regardless of the result, in order to encourage exploration. A discount factor of $\gamma = 0.97$ is used along with $\epsilon = 0.01$. This gives a horizon length of around 150 steps. The exploration constant $c = 0.1$ was experimentally determined to yield good results.

For both domains, *POrTAL* is given a constant reward of -1.0 per action, with no discounting. An exploration value of $c = 20$ is used. For the EXPAND procedure, the formula $G(h) < k \cdot N(h)^{\alpha}$ is used (taken from the literature on progressive widening), with $k = \frac{1}{2}$ and $\alpha = 1$. These values were experimentally determined. For the APPROXIMATE procedure, nodes are approximated as the mean value of their neighbor nodes within the tree, or zero if they have no neighbors. We believe that that domain-specific heuristics would produce better results.

We conducted two evaluations: an *Algorithm Comparison* and a *Time Comparison*. The *Algorithm Comparison* compares *POrTAL* to POMCP and FF-Replan baselines. *POrTAL* and POMCP are split into two additional variants, one in which the robot is given four seconds to plan per action, and another in which the robot is given 16 seconds to plan per action. We additionally vary the amount of uncertainty in the scene from from 2 to 10, incremented by 2. The amount of uncertainty refers to the number of possible locations that each item can exist within.

The *Time Comparison* is meant to show how *POrTAL* performs if given increasing timeout thresholds, starting from 2 seconds to action and doubling until 32 seconds per action. We compare two configurations of *POrTAL*: one in which there are 4 possible locations per item and another in which there are 8 possible locations per item.

C. Varying the Uncertainty

All of our evaluations involve varying the amount of uncertainty in each domain. We vary uncertainty in three ways: the *amount* of uncertainty, the *distribution of locations*, and the *distribution of likelihood*.

a) Amount of Uncertainty: This refers to the number of possible locations that an item can exist in. The number of locations is set to be the same for all items. For example, each item might be in two different locations, four different locations, and so on.

b) Distribution of Locations: This refers to choosing the sets of candidate locations that each item can possibly exist in. Sets are populated independently for each item, and the locations within each set are chosen uniformly without replacement. This means that two different items can be in

the same location, and that each item has the same number of distinct possible locations.

c) Distribution of Likelihood: In addition to varying the number of possible locations and the identities of these locations, we also varied the probability of items existing at these locations. We tested three different probability distributions: the *uniform* distribution, where each location has an equal chance of the item being there; a gradual *Decay-75* distribution, where each location ordered by increasing probability has 75% of the probability of an item being at that location as the next location; and a sharper *Decay-50* distribution, where each location has 50% of the probability as the next location of an item being at that location.

D. Results

Figure 3 (left) depicts our results for the *Algorithm Comparison*. Figure 3 (right) depicts our results for the *Time Comparison*. Our measure of performance is the number of steps required for the robot to achieve its goal, in which a lower amount of steps is better. We categorize our findings and discuss each category below.

*a) POrTAL **Often** Outperforms Baselines:* The *Algorithm Comparison* (Figure 3, left and center) shows that *POrTAL* more strongly outperforms POMCP under sharper likelihood distributions (see Section §IV-C.0.c) and less strongly outperforms—even sometimes underperforming—POMCP under more uniform likelihood distributions. Even so, for more uniform distributions, POMCP only begins to outperform *POrTAL* as the amount of uncertainty in the environment increases. **Implications:** We draw two conclusions from this result. First, *POrTAL* is more reliable than POMCP at producing good plans with low amounts of uncertainty. Second, *POrTAL* is better at producing plans in environments where uncertainty is concentrated more highly within specific locations, rather than being distributed evenly.

POrTAL is also shown to perform better than FF-Replan in more uniform distributions, and either better than or equal to FF-Replan in sharper distributions. **Implications:** We thereby conclude that *POrTAL* is less prone to bouncing back and forth between different candidate locations when searching for items, a behavior that FF-Replan exhibits. Unlike FF-Replan, *POrTAL* is able to search for objects at less-likely locations that are on the path towards to more-likely locations.

b) POrTAL is Lightweight: The 4-second variant of *POrTAL* outperforms the 16-second variant of POMCP in both office and elevator *Decay-50* distributions, the office *Decay-75* distribution, and a substantial portion of the office *Uniform* distribution. In these scenarios, *POrTAL* requires less planning time to achieve better results that POMCP. **Implications:** *POrTAL* is lightweight. In certain situations, *POrTAL* is more lightweight than POMCP in that it requires less planning time to produce better results.

c) Planner Performance Improves with Time: In the *Algorithm Comparison* (Figure 3, left), the 16-second variant of *POrTAL* and POMCP perform either better than or equal to their 4-second counterparts. The *Time Comparison* (Figure 3, right) similarly indicates that *POrTAL* improves with greater time allocated to the robot per action. **Implications:** While greater performance over time is a known phenomenon with POMCP, these result enables us to similarly conclude that *POrTAL* performs better over time.

d) Increasing the Timeout Yields Diminishing Returns: Both time comparisons (Figure 3, right) indicate that increasing the amount of time that the robot is allowed to plan per action beyond a certain point yields diminishing returns. **Implications:** Our results indicate that, on our hardware, this point is between 10 and 20 seconds. Before this point, increasing the timeout value drastically improves the robot's performance; after this point, performance improvements are minimal.

V. Discussion

For contexts that require high-performing and lightweight probabilistic planning solutions, *POrTAL* shows promise. In theory, *POrTAL* reduces the breadth of search by focusing search on the best possible sequences of actions. Our empirical results indicate that *POrTAL* produces good plans, often with less time than would be required from POMCP. Our results also indicate that, similar to POMCP, *POrTAL* produces better plans if allowed more planning time.

A. Limitations and Future Work

Our research has a few notable limitations. First, we have not yet tested *POrTAL* on a robot platform in a real-world context. Testing onboard a robot would enable us to determine realistic values for our testing configuration, including how much time the robot has available to plan per action. Testing in a real-world context would enable us to determine other parameters, such as the amount of uncertainty that is realistic in different environments and how this uncertainty is distributed. Lastly, testing on a robot platform would provide information on how *POrTAL* with limited compute resources, such as those available on the robot itself.

As a related limitation, we make certain assumptions that may not be realistic for physical robots. For example, our planning approach requires that the robot performs a dedicated *search* action in order to resolve uncertainty. In a real robot deployment, however, the robot will continuously observe new information passively. While the chances of observing new information might be higher with a dedicated *search* action, there is likely a non-zero probability of observing new information at any given point in time. Another assumption is that each step takes the same amount of time for the robot to complete. Future work is thereby required to test these assumptions on a robot platform.

Additionally, *POrTAL* is not optimal in guiding the robot to achieve its goal under the shortest amount of steps. This limitation is elevated in certain domains. For example, *POrTAL* will not perform well in certain domains that require the robot to plan for contingencies. One such domain involves the robot fetching an item from either side of a wall. If the robot travels to one side of the wall and discovers that

the item is not there, it might have a contingency plan in mind; for example, it can use a key to unlock a door and quickly retrieve the item from the other side. If the robot does not already have the key, *POrTAL* will not guide the robot to fetch it. With *POrTAL*, the robot will therefore have to traverse all the way around the wall to retrieve the item.

Related to the above limitation, *POrTAL* has only been tested under a small number of scenarios—the *office* and *elevator* scenario. Future work is required to test *POrTAL* under a greater number of representative scenarios in robotics.

VI. Conclusion

We present *POrTAL*, a probabilistic planning algorithm that draws inspiration from two existing baselines, POMCP and FF-Replan. *POrTAL* produces a search tree similar to POMCP, but draws from FF-Replan's determinization step in order to narrow the breadth of search and focus on useful paths. We evaluated *POrTAL* against both baselines and found that *POrTAL* often produces plans with a smaller number of steps, is more lightweight than baselines in the domains that we tested it within, and improves with greater amounts of time allocated for planning.

Acknowledgment

This research was supported by the U.S. Naval Research Laboratory. The views and conclusions contained in this document are those of the authors and should not be interpreted as necessarily representing the official policies, either expressed or implied, of the U.S. Navy.

References

[1] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.

[2] Shenghui Chen, Ruihan Zhao, Sandeep Chinchali, and Ufuk Topcu. Human-agent coordination in games under incomplete information via multi-step intent. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, AAMAS '25, page 481–489, Richland, SC, 2025. International Foundation for Autonomous Agents and Multiagent Systems.

[3] Maria Fox and Derek Long. Pddl2. 1: An extension to pddl for expressing temporal planning domains. *Journal of artificial intelligence research*, 20:61–124, 2003.

[4] Malik Ghallab, Dana Nau, and Paolo Traverso. *Automated planning and acting*. Cambridge University Press, 2016.

[5] Jörg Hoffmann, Julie Porteous, and Laura Sebastia. Ordered landmarks in planning. *Journal of Artificial Intelligence Research*, 22:215–278, 2004.

[6] Peter Karkus, David Hsu, and Wee Sun Lee. Qmdp-net: Deep learning for planning under partial observability. *Advances in neural information processing systems*, 30, 2017.

[7] Abhish Khanal, Joseph Prince Mathew, Cameron Nowzari, and Gregory J Stein. Learning-augmented model-based multi-robot planning for time-critical search and inspection under uncertainty. *arXiv preprint arXiv:2507.06129*, 2025.

[8] Andrey Kolobov, . Mausam, and Daniel Weld. Lrtdp versus uct for online probabilistic planning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 26(1):1786–1792, Sep. 2021.

[9] Iain Little, Sylvie Thiebaux, et al. Probabilistic planning vs. replanning. In *ICAPS Workshop on IPC: Past, Present and Future*, pages 1–10, 2007.

[10] Michael L Littman, Anthony R Cassandra, and Leslie Pack Kaelbling. Learning policies for partially observable environments: Scaling up. In *Machine Learning Proceedings 1995*, pages 362–370. Elsevier, 1995.

[11] Abhishek Paudel and Gregory J. Stein. Deployment-time selection of prompts for LLM-informed object search in partially-known environments. In *ICRA 2025 Workshop on Foundation Models and Neuro-Symbolic AI for Robotics*, 2025.

[12] David Porfirio, Mark Roberts, and Laura M. Hiatt. Uncertainty expression for human-robot task communication. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, AAMAS '25, page 1698–1707, Richland, SC, 2025. International Foundation for Autonomous Agents and Multiagent Systems.

[13] Silvia Richter and Matthias Westphal. The lama planner: Guiding cost-based anytime planning with landmarks. *Journal of Artificial Intelligence Research*, 39:127–177, 2010.

[14] David Silver and Joel Veness. Monte-carlo planning in large pomdps. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23. Curran Associates, Inc., 2010.

[15] Adhiraj Somani, Nan Ye, David Hsu, and Wee Sun Lee. Despot: Online pomdp planning with regularization. *Advances in neural information processing systems*, 26, 2013.

[16] Sungwook Yoon, Alan Fern, and Robert Givan. Ff-replan: a baseline for probabilistic planning. In *Proceedings of the Seventeenth International Conference on International Conference on Automated Planning and Scheduling*, ICAPS'07, page 352–359. AAAI Press, 2007.